

Assignment 1: Data Preparation

Nicholas Boidi, Pablos Tselioudis Garmendia, and Filippo Donghi

Data Mining Techniques, Vrije Universiteit Amsterdam
Group number: 65. Student numbers: 2892127, 2891511, 2888236.

1 Introduction

This report documents the work completed for Assignment 1 on the ODI-2026 survey dataset. We cover all five tasks: Task 1A (Exploratory Data Analysis), Task 1B (Data Cleaning and Imputation), Task 1C (Feature Engineering), Task 2A (Application of Classification Algorithms), Task 2B (Winning Classification Algorithms), Task 3 (Association Rules), Task 4 (Numerical Prediction), and Task 5 (Evaluation). The classification target is the binary variable `ir_course` and all modeling steps are described in a reproducible way.

2 Task 1A: Exploratory Data Analysis

The raw dataset contains 156 responses and 16 columns. After standardizing the column names, the timestamp was parsed as a datetime value and the remaining variables were treated according to their type. The dataset mixes categorical survey answers, free text, and several numeric responses that were not always entered in a structured form.

The programme field was highly heterogeneous, so spelling variants and aliases were grouped into broader categories. After grouping, the distribution is concentrated in a small number of study tracks: AI (50), Computer Science (30), Finance / Econometrics (21), Business Analytics (19), Bioinformatics (16), Other (12), and Computational Science (8).

Table 1. Summary of the numeric attributes after light cleaning.

Attribute	N valid	Invalid/outliers	Mean	Std	Median	Min	Max
Stress (0-100)	152	4	47.29	29.60	50.0	0.0	100.0
Sport hrs/week	155	1	4.65	3.67	4.0	0.0	20.0
Students est.	155	1	205.80	104.81	200.0	65.0	500.0
Random (1-10)	28	128	6.11	2.69	7.0	1.0	10.0

The numeric summary in Table 1 highlights three different patterns. Stress values cover the full 0–100 scale and are fairly dispersed. Sport hours are much more concentrated, with a low median of 4 hours per week. The student-estimate question is centered around 200, which is well below the actual room size of

roughly 500, so the class was systematically underestimated. The random-number question is the clearest anomaly: only 28 of 156 responses fall in the requested 1–10 interval, which shows that the column is dominated by noisy or playful entries.

Table 2. Unknown or ambiguous values after normalizing the categorical answers.

Variable	Unknown values
ML course	1
IR course	15
Databases	3
LLM used	0

The yes/no course questions were normalized to a common representation, while refusal phrases such as "not willing to say" were preserved as missing or unknown values. Table 2 shows that the information loss is small for machine learning and databases, but more substantial for information retrieval.

Figure 1 combines the main exploratory views. The programme distribution is imbalanced, with a few dominant categories and a long tail of smaller ones. The stress histogram spans the full scale, while sport hours remain concentrated near the lower end. The course-background bars show that most answers are well populated once the binary labels are normalized.

Figure 2 suggests a weak negative relationship between sport hours and stress. The overall Pearson correlation between the two variables is about -0.28 , so the trend is present but modest. The gender-specific fit lines follow the same direction, but the scatter is wide enough that the relationship is not strong enough to support a more ambitious claim.

Figure 3 shows that the student estimates cluster around 200, with the median exactly at 200. Since the actual classroom size was roughly 500, the responses indicate a consistent underestimation rather than a broad spread around the truth.

3 Task 1B: Data Cleaning and Imputation

The cleaning script standardized the raw survey answers before imputation. Text fields were stripped of extra whitespace, binary answers were normalized to yes/no, the programme field was mapped to a smaller set of semantic categories, and the timestamp was parsed into a datetime column. Structured numeric fields were parsed separately for birth year, room estimate, stress, sports hours, random number, and bedtime.

We then applied domain rules to remove clearly invalid values. Stress values outside 0–100 were set to missing, negative sport values were removed, bedtime values were constrained to the 0–24 range, and negative room estimates were discarded. In addition, room estimate and sports hours were passed through

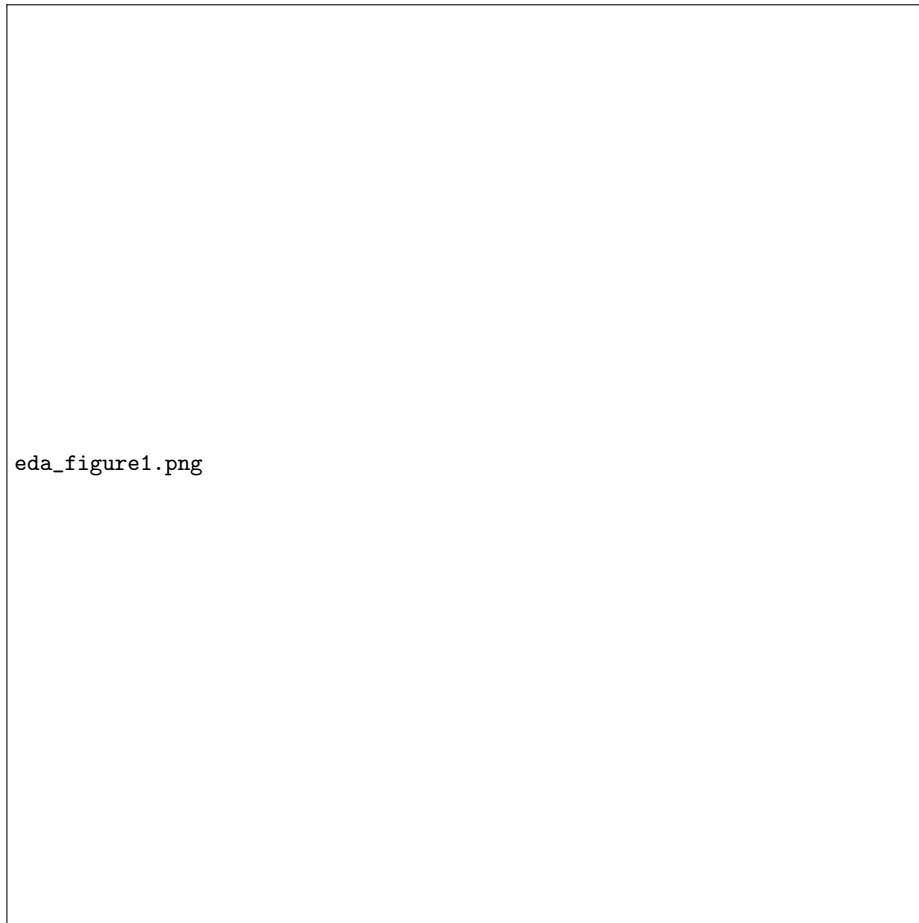


Fig. 1. Programme distribution, stress, sport hours, and course background after light cleaning.



Fig. 2. Stress versus sport hours, split by gender and fitted with simple linear trends.

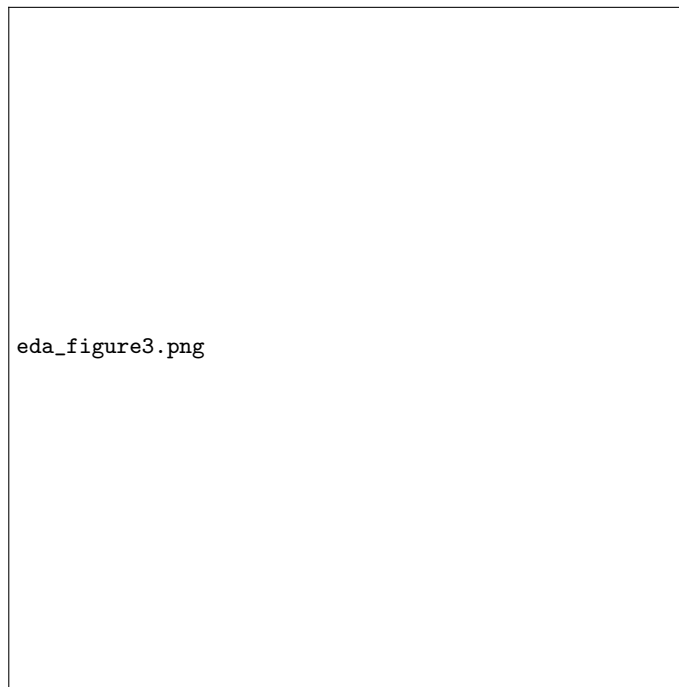


Fig. 3. Distribution of the estimated number of students in the room.

an IQR-based filter to remove extreme outliers. After these steps, the cleaned dataset retained the original row count but had fewer unreliable field values.

The imputation stage compared two standard approaches. The first uses median imputation for numeric variables and most-frequent imputation for categorical and free-text variables. This is a conservative baseline and is robust to skewed distributions and outliers [1]. The second uses KNN imputation with five neighbors and distance weighting for numeric variables, while keeping the same categorical/text fallback. KNN imputation can preserve local structure in the data better than a single global statistic [2].

Table 3. Missing values before and after the two imputation strategies.

Variable	Before	After simple	After KNN
Birth year	38	0	0
IR course	15	0	0
Sports hours	10	0	0
Statistics	10	0	0
Room estimate	8	0	0
Stress	5	0	0
Databases	3	0	0
Random number	3	0	0
Bedtime	3	0	0
ML course	1	0	0
Good day 1	1	0	0
Good day 2	1	0	0

Table 3 shows the missingness pattern before imputation. Birth year is by far the sparsest field, followed by information retrieval and sports hours. Both imputation strategies remove all missing values from the exported datasets, and the current code keeps both cleaned versions on disk for later comparison.

4 Task 1C: Feature Engineering

Feature engineering was applied to the cleaned dataset (`clean_imputed_simple.csv`) with the goal of enriching the input variables available for classification. We chose structured and interpretable features because the dataset is relatively small and consists mainly of survey variables. The aim was to create compact and meaningful predictors without introducing unnecessary complexity.

We introduced four new features. First, `age` was derived from `birth_year` using the formula $2026 - \text{birth_year}$, obtaining a more interpretable demographic variable than the raw birth year. Second, `courses_count` was defined as the number of technical courses taken by each student, counting `ml_course`, `stats_course`, and `db_course`, in order to summarize academic background in a single variable. Note that `ir_course` was deliberately excluded from this count

because it was selected as the classification target; including it would have introduced target leakage. Third, `late_bed` was constructed as a binary indicator of whether a student reported going to bed after 1:30 AM, simplifying the raw bedtime variable into a clearer behavioral pattern. Finally, `sport_stress_ratio` was defined as `sports_hours/(stress+1)`, combining physical activity and perceived stress into a single behavioral feature. The +1 in the denominator prevents division by zero when stress is reported as zero.

Together, these four features combine demographic, academic, and behavioral information into compact representations that may help the model distinguish students who took the IR course from those who did not.

5 Task 2A: Application of Classification Algorithms

For the classification task we use the cleaned basic dataset `task1b/clean_imputed_simple.csv` with 156 instances. The class to predict is `ir_course`, mapped to a binary target (yes=1, no=0). The class distribution is mildly imbalanced (58 yes, 98 no), so we evaluate performance primarily with the F1-score, which balances precision and recall and is more informative than plain accuracy under imbalance [6].

Reproducible Pipeline

All experiments were run with a fixed random seed (42) in script `task2a/task_2a.py`. We used a stratified train/test split (80%/20%), yielding 124 training and 32 test instances.

The preprocessing and modeling pipeline was implemented with scikit-learn [3]. We derived two numeric timestamp features (hour and weekday), removed free-text fields (`good_day_1`, `good_day_2`), one-hot encoded categorical attributes, and applied median/most-frequent imputation where needed. Numeric features were standardized to support distance-based learning.

We compare two classification algorithms discussed in class:

- Decision Tree classifier [4]
- k-Nearest Neighbors (k-NN) [5]

Hyperparameter Optimization

Hyperparameters were tuned with grid search and 5-fold stratified cross-validation on the training set, optimizing F1-score.

Results

Table 5 reports both cross-validation performance (on training folds) and held-out test performance.

The decision tree achieved higher cross-validated F1 during tuning, but generalized poorly to the held-out test set, suggesting overfitting on this small and

Table 4. Hyperparameter search spaces and selected configurations.

Model	Search space (main parameters)	Best configuration
Decision Tree	criterion {gini, entropy}; max_depth {None, 3, 5, 7, 10}; min_samples_split {2, 5, 10}; min_samples_leaf {1, 2, 4}	criterion=entropy; max_depth=10; min_samples_split=5; min_samples_leaf=1
k-NN	n_neighbors {3, 5, 7, 9, 11}; weights {uniform, distance}; p {1, 2}	n_neighbors=7; weights=uniform; p=1

Table 5. Task 2A performance for tuned classifiers (target: `ir_course`).

Model	CV F1	Test Acc.	Test Prec.	Test Rec.	Test F1	Test ROC-AUC
Decision Tree	0.553	0.438	0.200	0.167	0.182	0.373
k-NN	0.485	0.500	0.333	0.333	0.333	0.563

noisy dataset. The tuned k-NN model is more stable on the test split and provides the better final F1-score (0.333 vs. 0.182), so we select k-NN as the preferred classifier for this target under the current feature set.

From an interpretation perspective, both models indicate that predicting `ir_course` from the available survey variables is non-trivial. The low absolute scores suggest that additional feature engineering (Task 1C extensions), stronger class-balancing strategies, or alternative model families may be needed to improve performance substantially.

6 Task 2B: Winning Classification Algorithms

To study a high-performing real-world classification approach, we analyzed the winner of the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2015 classification track [7]. Although Kaggle and KDD Cup are common platforms, ILSVRC is also a landmark machine learning competition and a strong example of how winning approaches differ from standard baselines.

Competition Description

ILSVRC 2015 is a large-scale image classification competition with 1,000 classes and more than one million training images from the ImageNet dataset [7]. The core task is multi-class classification: assign each image to the correct object class. The primary evaluation measure is top-5 classification error, i.e., an image is counted as correctly classified if the true class appears in the model’s five highest-confidence predictions.

Winner and Technique

The winner of ILSVRC 2015 classification was the Microsoft Research team, using the Residual Network (ResNet) architecture [8]. Their model family (up to 152 layers) achieved state-of-the-art accuracy at the time and significantly improved over earlier deep convolutional networks.

Main Idea of the Winning Approach

The key idea is residual learning. Instead of learning a direct mapping $H(x)$, each residual block learns a residual function $F(x) = H(x) - x$ and outputs $F(x) + x$ through shortcut (skip) connections. This design improves gradient flow during training and makes very deep networks trainable without severe degradation problems. In practice, this allowed much deeper models than standard plain CNN stacks while maintaining optimization stability [8].

Why It Stood Out

Compared with non-winning or standard architectures of that period, the winning approach stood out for three reasons:

- It enabled substantially deeper networks with better optimization behavior due to skip connections.
- It improved generalization and reduced classification error at large scale.
- It provided an architectural principle (residual blocks) that transferred well to many later tasks and became a default baseline in computer vision.

In short, the winning method was not only a parameter tuning improvement over standard CNNs; it introduced an architectural innovation that changed how classification models are designed and trained.

7 Task 3: Association Rules

Association rule mining, pioneered by the APRIORI algorithm [12], discovers frequent itemsets and generates rules like “customers who buy bread and butter also buy milk.” However, the raw association rules lack business hierarchy and context. One important category of improvements involves product hierarchies, also known as generalization or taxonomy-based association rules, which group products into higher-level categories [13].

Hierarchical Product Categorization

Instead of finding rules at the individual product level, hierarchical approaches aggregate products into multiple levels of abstraction. For example, in a grocery store, “cheese” and “yogurt” are both in the “dairy products” category, which sits under “food.” A rule mined at the product level might be “customers who buy Feta cheese also buy Greek yogurt” (highly specific, rare), while at the category level it becomes “customers who buy dairy products also buy bread” (broader, more frequent).

The process works as follows: First, define a product hierarchy (e.g., product $\rightarrow$ category $\rightarrow$ department $\rightarrow$ section). Second, apply APRIORI or similar mining algorithms not just at the product level, but also at each level of the hierarchy. Third, report rules at all levels, allowing stakeholders to choose the appropriate granularity for their decision-making [13].

Advantages

- **Fewer rare itemsets:** General categories appear more frequently than specific products, raising support thresholds and filtering out noise [13].
- **More actionable insights:** Rules at category level guide merchandising without product-specific decisions.
- **Multiple levels of analysis:** Stakeholders can drill down from high-level to low-level rules as needed.
- **Scalability improvements:** Mining fewer, more frequent itemsets is computationally cheaper [13].
- **Domain knowledge integration:** The hierarchy encodes expert knowledge about product relationships.

Disadvantages

- **Information loss:** Grouping products erases fine-grained signals and niche market patterns.
- **Hierarchy design is critical:** Poor hierarchies introduce noise and miss natural patterns [13].
- **Increased complexity:** Running APRIORI at multiple levels is more complex than single-level mining.
- **Static hierarchy assumption:** Most approaches assume hierarchies are fixed and known in advance.
- **Cross-level conflicts:** Product-level and category-level rules may contradict each other [13].

Hierarchical association rules are most effective for large product spaces where experts have defined meaningful hierarchies and business decisions naturally operate at category levels.

8 Task 4: Numerical Prediction

The goal of this task is to predict a continuous numerical target using two regression algorithms. The target variable chosen is **stress**, which represents self-reported stress levels on a scale from 0 to 100. This variable was selected because it has the widest range and highest variance among all numeric columns in the dataset. It also shows the strongest correlations with other features, particularly sports hours ($r = -0.26$) and bedtime ($r = -0.23$). The other numeric candidates like **birth_year** or **room_estimate** have almost no correlation with any feature, making them poor regression targets.

Setup

The cleaned dataset from Task 1B (`clean_imputed_simple.csv`) was used with 156 instances. Free-text columns (`good_day_1`, `good_day_2`) and the `random_number`

column were dropped. The random number column was removed because it contained extreme outlier values (up to 1.1×10^{37}) and carries no predictive signal for stress.

Categorical variables were preprocessed before modeling. The `programme` field was grouped to merge rare categories (fewer than 4 instances) into an "Other" label. Binary yes/no columns were encoded as 0/1, and the `stats_course` column (which uses "mu" and "sigma" labels) was converted to a binary indicator. The timestamp was split into hour and weekday features. All numeric features were standardized, and the remaining categorical features were one-hot encoded.

The data was split into 80% training (124 instances) and 20% test (32 instances) using a fixed random seed of 42 for reproducibility.

Regression Approaches

Two regression algorithms were compared:

- **Ridge Regression** [14]: a linear model with L2 regularization. It penalizes large coefficients, which helps prevent overfitting on small datasets with many features after one-hot encoding.
- **Random Forest Regressor** [15]: an ensemble of decision trees that captures non-linear relationships. Each tree is trained on a bootstrap sample, and the final prediction is the average across all trees.

Ridge was chosen as the linear baseline because standard linear regression tends to overfit when the number of features (after encoding) approaches the number of samples. Random Forest was chosen as the non-linear alternative because it handles mixed feature types well and does not assume a linear relationship between predictors and the target.

Hyperparameter Optimization

Hyperparameters were tuned using grid search with 5-fold cross-validation on the training set, optimizing for negative mean squared error.

Table 6. Hyperparameter search spaces and best configurations.

Model	Search space	Best configuration
Ridge	alpha {0.01, 0.1, 1.0, 10.0, 100.0}	alpha = 100.0
Random Forest	n_estimators {50, 100, 200} max_depth {None, 5, 10, 15} min_samples_split {2, 5, 10} min_samples_leaf {1, 2, 4}	n_estimators = 200 max_depth = 5 min_samples_split = 10 min_samples_leaf = 1

The high alpha value selected for Ridge (100.0) indicates that strong regularization is needed. This makes sense given the small dataset and the relatively

large number of features after one-hot encoding. For Random Forest, the shallow depth (5) and high split threshold (10) suggest the model also benefits from being constrained to avoid overfitting.

Results

Table 7. Task 4 regression performance for tuned models (target: `stress`).

Model	CV RMSE	Test MSE	Test RMSE	Test MAE	Test R^2
Ridge	28.25	691.32	26.29	22.52	0.116
Random Forest	28.24	825.01	28.72	25.18	-0.055

Ridge outperforms Random Forest on the test set across all metrics. The test R^2 of 0.116 for Ridge is low but positive, meaning it explains about 12% of the variance in stress levels. Random Forest achieves a negative R^2 , which means it performs worse than simply predicting the mean stress value for every instance.

Both models had nearly identical cross-validation RMSE (around 28.2), but their test set behavior diverged. This gap is likely caused by the small test set size (32 samples), where a few difficult instances can shift the metrics noticeably.

Interpretation

Figure 4 shows the actual versus predicted stress values for both models. Ridge predictions spread along a slight upward trend, indicating some ability to distinguish low-stress from high-stress students. Random Forest predictions cluster more tightly in the 30 to 60 range regardless of the actual value. This is a common pattern when a model defaults to predicting near the training set mean.

Figure 5 shows the distribution of residuals (actual minus predicted) for both models. Ridge residuals are spread roughly evenly around zero, while Random Forest residuals skew negative. The negative skew means the Random Forest tends to overpredict stress more often than it underpredicts it.

Comparing the two models directly, Ridge wins on every test metric. Its MAE is about 2.7 points lower than Random Forest, and its R^2 is positive while Random Forest's is negative. The high regularization strength selected for Ridge ($\alpha = 100.0$) effectively shrinks most coefficients toward zero, keeping only the features with the strongest signal. Random Forest, despite being more flexible, does not benefit from that flexibility here. With only 124 training samples, it lacks enough data to reliably learn non-linear splits across 5 levels of depth. The result is a model that memorizes training noise and then regresses toward the mean on unseen data.

The overall low performance of both models is expected. Stress is inherently subjective. Predicting it from a handful of survey demographics like programme, courses taken, and sport hours is a hard problem. The features available simply do not capture enough of what drives individual stress levels. That said,



Fig. 4. Actual vs predicted stress levels for Ridge and Random Forest.

Ridge still manages to pick up some weak linear signal, particularly from sports hours and bedtime, which is consistent with the correlations observed during exploratory analysis. Based on these results, Ridge is selected as the better model for this regression task.

Compared to the classification task in Task 2A, the regression problem proved harder. Classification only needed to separate two groups (took IR course or not), while regression requires predicting exact values on a continuous scale. A small dataset amplifies this difficulty because the model has fewer examples to learn subtle numeric patterns from. Both tasks share the same underlying limitation though: the survey features collected during the first lecture are simply not strong predictors of either course enrollment or stress.



Fig. 5. Residual distributions for Ridge and Random Forest.

9 Task 5: Evaluation

Task 5A: Characteristics of Evaluation Metrics

Mean Squared Error (MSE) and Mean Absolute Error (MAE) are two standard metrics for evaluating regression models. Their formulae are:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad \text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

where y_i is the actual value, $\hat{y}_i$ is the predicted value, and n is the number of samples.

The key difference between them comes down to how they treat large errors. MSE squares each error before averaging, which means a single prediction that

is off by 40 points gets penalized 16 times more than one that is off by 10. MAE treats both proportionally, so the same pair of errors contributes only 4 times more. In practice, this means MSE is more sensitive to outliers. If the goal is to minimize worst-case errors or if large mistakes are especially costly, MSE is the better choice. If all errors matter equally regardless of size, MAE gives a more representative picture of typical model performance.

There is also an optimization angle. MSE is differentiable everywhere, which makes it easier to use with gradient-based training methods. MAE has a non-differentiable point at zero, which can cause issues during optimization. This is one reason why MSE is more commonly used as a training objective even when MAE might be the preferred evaluation metric.

MSE and MAE produce identical results when every prediction error has exactly the same magnitude. Consider a simple example: a model predicts housing prices for 4 houses, and every prediction is off by exactly 10,000 euros. In that case, $MAE = 10,000$ and $MSE = 100,000,000$. The values differ, but both metrics agree perfectly on which model is better because neither is affected by variance in the error distribution. There is no variance. If a second model also makes uniform errors but of size 12,000, both MSE and MAE will rank the first model higher.

More formally, MSE can be decomposed as $MSE = MAE^2 + \text{Var}(\text{errors})$ when errors are all the same sign. When the variance of the error distribution is zero (all errors equal), the two metrics carry the same information and will always rank models identically. They start to disagree when one model has high error variance, meaning a mix of small and large mistakes, because MSE penalizes the large ones disproportionately while MAE does not.

Task 5B: Impact of Evaluation Metrics

Table 8 shows the MSE and MAE values for both regression models from Task 4.

Table 8. MSE and MAE comparison for Ridge and Random Forest on stress prediction.

Model	MSE	RMSE	MAE	Median AE	Max AE
Ridge	691.32	26.29	22.52	20.45	45.82
Random Forest	825.01	28.72	25.18	23.45	56.97

Ridge outperforms Random Forest under both metrics, but the gap is not the same. The MSE difference between the two models is 133.69 (a 19.3% increase from Ridge to Random Forest), while the MAE difference is only 2.66 (an 11.8% increase). MSE amplifies the gap because Random Forest makes a few large errors that get squared. Its worst prediction is off by nearly 57 points, compared to Ridge's worst at about 46 points. That single outlier error contributes disproportionately to MSE.

Figure 6 shows the per-sample absolute errors sorted from largest to smallest for both models. The darker bars represent absolute errors (what MAE averages), while the lighter overlay shows the squared errors scaled to the same axis. For both models, the top 5 largest errors account for roughly 40 to 43% of the total MSE but only about 29% of the total MAE. This demonstrates exactly how MSE amplifies the influence of outlier predictions.



Fig. 6. Per-sample errors sorted by magnitude, showing how large errors contribute disproportionately to MSE compared to MAE.

For this particular problem, MAE is arguably the more useful metric. Stress is self-reported on a subjective scale, so the data itself contains noise. A prediction that is off by 40 points is not necessarily four times worse than one that is off by 20, it could just reflect inconsistent self-reporting. MAE treats these errors more

evenly and gives a more intuitive summary: on average, the model's predictions are about 22 to 25 points away from the actual stress value. That is easy to interpret on a 0 to 100 scale.

That said, MSE still adds value as a complementary metric. The fact that Ridge has a noticeably lower MSE tells us it avoids the extreme mispredictions that Random Forest occasionally makes. If the application required reliable worst-case behavior, MSE would flag Random Forest's weakness more clearly than MAE does.

References

1. Little, R.J.A., Rubin, D.B.: Statistical Analysis with Missing Data. 2nd edn. Wiley, New York (2002)
2. Troyanskaya, O., Cantor, M., Sherlock, G., Brown, P., Hastie, T., Tibshirani, R., Botstein, D., Altman, R.B.: Missing value estimation methods for DNA microarrays. *Bioinformatics* **17**(6), 520–525 (2001)
3. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* **12**, 2825–2830 (2011)
4. Breiman, L., Friedman, J.H., Olshen, R.A., Stone, C.J.: Classification and Regression Trees. Wadsworth, Belmont (1984)
5. Cover, T., Hart, P.: Nearest neighbor pattern classification. *IEEE Transactions on Information Theory* **13**(1), 21–27 (1967)
6. van Rijsbergen, C.J.: Information Retrieval. 2nd edn. Butterworth-Heinemann, London (1979)
7. Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A.C., Fei-Fei, L.: ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision* **115**(3), 211–252 (2015)
8. He, K., Zhang, X., Ren, S., Sun, J.: Deep Residual Learning for Image Recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778 (2016)
9. Agrawal, R., Imieliński, T., Swami, A.: Mining association rules between sets of items in large databases. In: Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data, pp. 207–216 (1993)
10. Srikant, R., Agrawal, R.: Mining generalized association rules. In: Proceedings of the 23rd International Conference on Very Large Data Bases (VLDB), pp. 407–419 (1997)
11. Hoerl, A.E., Kennard, R.W.: Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics* **12**(1), 55–67 (1970)
12. Breiman, L.: Random forests. *Machine Learning* **45**(1), 5–32 (2001)